

Part 0 — Foundations

Revised Edition — incorporating a senior engineering / curriculum-design review

Everything to set up before Phase 1 begins: environment setup, raw Java syntax, the terminal, Git/GitHub, UML diagramming, and the dev-tools stack the rest of the roadmap assumes you already have running. This revision reorders topics so every prerequisite lands before the topic that depends on it, and fills gaps (exception handling, arrays, debugging, interactive input) that the original left out.

NEW IN THIS REVISION: This edition changed four things: (1) added a short **Environment Setup** topic before everything else, (2) moved **Terminal & Command Line** before **Java Anatomy & Syntax** since Java's own projects require it, (3) added missing subtopics — exceptions, arrays, Scanner input, debugging — to Java Anatomy, and (4) refreshed tools/resources for 2026 (GitHub CLI, SDKMAN, WSL2, AI coding assistants) and softened the Windows-CMD content to a secondary note rather than an equal track.

Topics	7 (was 6 — added Environment Setup)
Estimated Duration	2.5–3.5 weeks (was 2–3 — content added)
Difficulty	Beginner
Suggested Pace	8–12 hrs/week · 2–3 topics/week (part-time default)
Projects Included	4–6 detailed hands-on projects per topic, step-by-step
Documentation Sources	3+ official/primary sources per topic

What's Inside This PDF

- A short introduction to what this part covers and how it fits the full roadmap
- An at-a-glance table of duration, difficulty, and pacing
- The complete topic content: subtopics, concepts, and reference tables — reordered and gap-filled
- 4–6 detailed hands-on projects per topic with step-by-step instructions
- Official documentation links (Oracle, MDN, GitHub Docs, Baeldung, etc.) for every topic
- A standalone progress tracker checklist scoped to this part only

Topics Covered (Revised Order)

- **0. Environment Setup** — **NEW**
- 1. Terminal & Command Line Basics — **moved earlier**
- 2. Java Anatomy & Syntax — **expanded**
- 3. Git & GitHub Essentials
- 4. Git Commands Reference
- 5. UML Basics — **project 1 fixed**
- 6. Dev Tools & Apps Roundup — **refreshed for 2026**

Next: once every topic below is checked off, move on to Part A — Backend Engineering.

PART 0 · FOUNDATIONS

0 · Environment Setup

New topic · install the JDK and a shell before touching any of the topics below

NEW IN THIS REVISION: This topic didn't exist in the original roadmap — every other topic silently assumed the JDK, a terminal, and an IDE already worked. It goes first because nothing else in Part 0 is possible without it.

Subtopics covered: installing the JDK · verifying the install · JAVA_HOME · choosing one shell environment · installing IntelliJ IDEA

Install the JDK

- Install a current LTS JDK (Java 21 or later) — Eclipse Temurin or Oracle's own build both work
- On macOS/Linux, consider **SDKMAN** (sdkman.io) to install and switch JDK versions cleanly — this becomes useful fast once different projects need different versions
- On Windows, use the official installer, or WSL2 + SDKMAN for a Unix-style setup (recommended — see below)

Verify the Install

- `java -version` and `javac -version` should print matching version numbers
- If either command isn't found, the JDK's bin folder isn't on your PATH yet (this connects directly to the PATH subtopic in the next section)
- Confirm JAVA_HOME is set and points at the JDK install directory — several build tools (Maven/Gradle) read this variable directly

Choose One Shell Environment

- macOS and Linux: your default terminal (zsh/bash) is already a real Unix shell — nothing else to install
- **Windows: install WSL2** (Windows Subsystem for Linux) and do the rest of this roadmap inside it. This is the standard 2026 setup for Windows developers doing backend/Java work — it gives you a real Linux environment instead of translating every command between two syntaxes
- Git Bash remains a lighter-weight fallback if WSL2 isn't an option, but WSL2 is the better long-term investment since Docker (Part C) expects a Linux-like environment anyway

Install IntelliJ IDEA

- IntelliJ IDEA Community Edition is free and sufficient for this entire roadmap
- Confirm it can open a single .java file and run it before moving on — this is also where you'll first see the built-in debugger, used starting in the next topic

1 First Boot Checklist

Steps: install the JDK → run `java -version` and `javac -version` and confirm they match → confirm JAVA_HOME is set → install IntelliJ IDEA → open a blank project and confirm the Run button works on a one-line program.

Study Resources

- Oracle: JDK Installation Guide — <https://docs.oracle.com/en/java/javase/21/install/>
- SDKMAN — <https://sdkman.io/>
- Microsoft: WSL Installation Guide — <https://learn.microsoft.com/en-us/windows/wsl/install>

Before moving on, you should be able to: open a terminal and run `java -version` successfully; explain what `JAVA_HOME` is used for; and open and run a one-file Java program in IntelliJ.

PART 0 · FOUNDATIONS

1 · Terminal & Command Line Basics

Navigating without a mouse · running your own programs · one primary shell, not two

NEW IN THIS REVISION: Moved ahead of Java Anatomy & Syntax: the Java topic's own projects (Command-Line Calculator, Package Practice) already require running `javac` and `java` from a terminal, so this needs to come first, not second.

Every tool from here on — `javac`, `git`, `npm`, `docker`, `mvn` — is run from a terminal. If typing commands still feels unfamiliar, this is the section to slow down on; everything downstream assumes it's second nature.

Subtopics covered: navigation · file operations · running programs · environment variables & `PATH` · one primary shell (Unix/WSL2), Windows CMD as a brief note only

Navigation

- `pwd` — print the current directory
- `cd foldername` to move in, `cd ..` to move up, `cd ~` to jump home
- `ls` to list contents; `ls -la` for hidden files and details

File & Folder Operations

- `mkdir` to create a folder, `touch file.txt` to create an empty file
- `cp` to copy; `mv` to move or rename
- `rm` to delete a file; `rm -r` for a folder

■ **Careful:** `rm` and `rm -r` have no undo, no trash bin, no confirmation prompt by default. Double-check the path before you hit enter.

Running Programs From the Terminal

- `javac File.java` to compile, `java File` to run a compiled Java class
- `node file.js` to run a JavaScript file directly with Node.js
- Running a project's start script: `npm run dev`, `mvn spring-boot:run` — you'll use these for real once Part B (frontend) and Part A (Spring Boot) begin

Environment Variables & PATH

- What `PATH` is: the list of folders the shell searches for a command's executable
- `echo $PATH` to view it; why 'command not found' usually means a `PATH` issue — this is exactly what would happen if the JDK setup in the previous topic wasn't on `PATH`

- Setting a variable for one session (`export VAR=value`) vs. permanently (in your shell profile file, e.g. `.zshrc`) — this pattern reappears later with `.env` files and Docker environment variables

A Note on Windows

If you set up WSL2 in the Environment Setup topic, you're already using a real Unix shell and everything above applies directly. If you're on plain Windows CMD/PowerShell instead, the equivalents are: `dir` for `ls`, `del` for `rm`, copy/move the same as Unix, and `echo %PATH%` for viewing `PATH`. Treat this as a fallback, not a parallel track to master — the rest of this roadmap, and virtually all professional Java/Spring tooling, assumes a Unix-style shell.

Suggested Project — Detailed Instructions

1 Folder Structure Builder

Steps: `mkdir` a project folder → `cd` into it → `mkdir` `src`, `test`, and `docs` subfolders → touch a `README.md` inside it → `ls -la` to confirm the structure

2 Compile-and-Run Drill

Steps: write a one-file `HelloTerminal.java` → `cd` to its folder → `javac HelloTerminal.java` → `java HelloTerminal` → confirm the `.class` file appeared with `ls`

3 PATH Investigation

Steps: run `which java` → print `PATH` and locate that folder in the list → temporarily remove it from `PATH` in one session → confirm `java` stops being found → restore `PATH`

4 Batch File Cleanup

Steps: create 10 dummy files with mixed extensions → list only `.txt` files with a wildcard → move all `.txt` files into a new subfolder → delete the leftovers → verify with `ls`

5 Shell Profile Customization

Steps: find your shell profile file (`.zshrc`/`.bashrc`) → add an alias like `ll` for `ls -la` → reload the profile → confirm the alias works in a new terminal tab

Study Resources

- MDN: Command line crash course — https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Environment_setup/Command_line
- Microsoft: WSL documentation — <https://learn.microsoft.com/en-us/windows/wsl/>
- GNU Coreutils Manual (Unix commands, official) — <https://www.gnu.org/software/coreutils/manual/>

2 · Java Anatomy & Syntax

The building blocks · read before you write your first real program

NEW IN THIS REVISION: Expanded from the original: added **arrays** as a real subtopic instead of a passing mention, added **exception handling** (missing entirely before), added **Scanner input** so students aren't limited to args[]-only programs, and added a **debugging** subtopic so println isn't the only debugging tool taught.

Before OOP, make sure the raw syntax is second nature. This section exists so a line like `public static void main(String[] args)` stops looking like a magic incantation and starts looking like five separate, understandable decisions.

Anatomy of a Java Program

```
public class HelloWorld { public static void main(String[] args) {
System.out.println("Hello"); } }
```

Token	What it means
public	Access modifier — this class/method is visible from anywhere, including other packages
class HelloWorld	Declares a class named HelloWorld; the filename must match this name exactly (HelloWorld.java)
static	Belongs to the class itself, not to any instance — the JVM can call main() without creating an object
void	This method returns no value
main	The exact method name the JVM looks for as the program's entry point
(String[] args)	An array of command-line arguments passed in when the program is run; empty if none are given
System.out.println(...)	Calls println on the standard output stream to print a line of text

Subtopics covered: keywords & access modifiers · naming conventions · variables & primitive types · **arrays** · operators · control flow · **exception handling** · **Scanner & console input** · methods · compiling, running & packages · **reading stack traces & using a debugger**

Keywords & Access Modifiers

- Access modifiers: public, private, protected, and default (package-private, no keyword written)
- static vs instance: static belongs to the class; instance members need an object created with new
- final: locks a variable's value, a method against overriding, or a class against subclassing
- class / interface / extends / implements: the vocabulary for building type hierarchies (Phase 1)

Naming Conventions

- Classes & interfaces: PascalCase — Car, PaymentProcessor, Comparable
- Methods & variables: camelCase — getBalance(), totalPrice, isActive
- Constants (static final): UPPER_SNAKE_CASE — MAX_USERS, DEFAULT_TIMEOUT
- Packages: all lowercase, reverse-domain style — com.shopflow.service

Variables & Primitive Types

- Primitives: int, long, double, float, boolean, char, byte, short — stored by value
- Reference types: String, arrays, and any object — stored as a reference to memory
- Type declaration syntax: `type name = value`; e.g. `int age = 20`;
- `var` (local type inference, Java 10+): lets the compiler infer the type from the right-hand side

Arrays

GAP FILLED: Added — the original only mentioned arrays in one line, yet its own Command-Line Calculator project already indexes into `args[]`.

- Declaration: `int[] nums = new int[5]`; or with a literal: `int[] nums = {1, 2, 3}`;
- Zero-indexed: `nums[0]` is the first element; `nums.length` gives the size
- Iterating: a standard for-loop with an index, or an enhanced for-each when you don't need the index
- Arrays have a fixed size once created — this is exactly why Phase 1's collection classes (`ArrayList`, etc.) exist

Operators

- Arithmetic: `+` `-` `*` `/` `%` ; integer division vs floating-point division
- Relational: `==` `!=` `>` `<` `>=` `<=` ; note `==` compares references for objects, not content (use `.equals()`)
- Logical: `&&` `||` `!` and short-circuit evaluation
- Assignment shorthand: `+=` `-=` `*=` `/=` `++` `--`

Control Flow

- `if` / `else if` / `else`, and the ternary operator (`condition ? a : b`)
- `switch` statements and modern `switch` expressions (Java 14+, arrow syntax) — **prefer the arrow syntax going forward**; recognize the older colon-style syntax only because you'll see it in existing codebases
- Loops: `for`, enhanced `for-each`, `while`, `do-while` — when to reach for each
- `break` / `continue`, and labeled loops for nested breaks

Exception Handling

GAP FILLED: Added — this was missing entirely from the original, despite being core Java syntax rather than a later-phase topic.

- `try` / `catch` / `finally`: isolate code that might fail and handle the failure without crashing the program
- Checked exceptions (must be declared or caught, e.g. `IOException`) vs. unchecked exceptions (`RuntimeException` and its subclasses, e.g. `NullPointerException`, `ArithmeticException`)
- `throw` to raise an exception yourself; `throws` in a method signature to declare one it might produce
- Writing a simple custom exception class by extending `Exception` or `RuntimeException`

Scanner & Console Input

GAP FILLED: Added — the original's only input mechanism was `args[]`; this adds interactive, in-program input.

- `Scanner sc = new Scanner(System.in)`; reads from standard input

- `sc.nextLine()`, `sc.nextInt()`, `sc.nextDouble()` for typed input
- Basic input validation: checking with `hasNextInt()` before consuming, and catching `InputMismatchException`

Methods

- Declaration syntax: `modifier returnType methodName(paramType paramName) { ... }`
- Method overloading: same name, different parameter lists
- Passing by value: primitives are copied; object references are copied (but point to the same object)

Compiling, Running & Packages

- JDK vs JRE vs JVM: JDK to write/compile code, JRE to run it, JVM as the engine that executes bytecode
- `javac HelloWorld.java` compiles to `HelloWorld.class`; `java HelloWorld` runs it
- package declaration at the top of a file, and import to use classes from other packages
- Comments: `//`, `/* */`, and `/** */` Javadoc comments for documenting public APIs

Reading Stack Traces & Using a Debugger

GAP FILLED: Added — arguably the single most-used beginner survival skill, and it was absent from the entire original roadmap.

- Reading a stack trace top-down: the exception type and message, then the exact file and line where it was thrown, then the call chain that led there
- Common early exceptions to recognize on sight: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`, `ClassCastException`
- Setting a breakpoint in IntelliJ IDEA and running in Debug mode instead of Run mode
- Stepping over vs. stepping into a method call, and inspecting variable values in the debugger panel while paused

Suggested Project — Detailed Instructions

1 Hello World Variants

Steps: declare one variable per primitive type → print each with a label → combine with `String.format()` → run with `java` and confirm output

2 Syntax Playground

Steps: write an `if/else` chain → add a `switch` expression → add a `for`, a `while`, and a `do-while` loop → add one labeled `break` in a nested loop

3 Array & Scanner Quiz App [prompt the user with Scanner for 3 quiz questions → store answers in a `String[]` array → compare against a correct-answers array → print a score out of 3]

Steps: NEW

4 Overload Practice

Steps: write `add(int,int)` → write `add(double,double)` → write `add(String,String)` for concatenation → call all three from `main()` and print results

5 Command-Line Calculator (with real error handling) [parse args[0] and args[2] with Integer.parseInt → read the operator from args[1] → switch on the operator → wrap the divide operation in try/catch for ArithmeticException instead of a manual zero-check → also catch NumberFormatException around the parsing]

Steps: REVISED

6 Stack Trace Autopsy [deliberately write code that throws a NullPointerException, then a ArrayIndexOutOfBoundsException → read each stack trace top to bottom → identify the exact failing line before fixing it → repeat the same exercise using the IntelliJ debugger with a breakpoint instead of just reading output]

Steps: NEW

Study Resources

- Oracle: The Java Tutorials — <https://docs.oracle.com/javase/tutorial/>
- Oracle: Java Language Basics — <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html>
- dev.java (official Java developer site) — <https://dev.java/learn/>
- Baeldung: Java Exceptions — <https://www.baeldung.com/java-exceptions>
- Java SE API Documentation (Javadoc) — <https://docs.oracle.com/en/java/javase/21/docs/api/index.html>

Before moving on, you should be able to: read any short Java method and explain what every keyword does; declare and iterate over an array; write an if/else, a switch, and all three loop types from memory; wrap risky code in try/catch and explain the difference between checked and unchecked exceptions; read a stack trace and find the failing line without help; and explain the difference between == and .equals() for objects.

3 · Git & GitHub Essentials

Version control · collaboration · the workflow every dev job runs on

NEW IN THIS REVISION: Content unchanged from the original — it was already strong. Added a priority split so beginners know which concepts to master now vs. revisit later, plus a warning on the cheat sheet's git reset --hard row.

You'll commit your very first Phase 1 mini-project to Git, so this comes before OOP, not after. Each topic below explains what it is, why it matters, and how it actually shows up in a company — not just the command syntax.

Master these now: init/clone, add, commit, push/pull, branch, merge, pull requests. Come back to these once the basics feel natural: rebase, cherry-pick, choosing a branching strategy for a whole team.

Subtopics covered: commit lifecycle · branching · merge vs rebase · conflict resolution · .gitignore & secrets · undoing changes · commit conventions · pull requests & review · issues & boards · branching strategy · branch protection · authentication

Git Basics & the Commit Lifecycle

Git is a distributed version control system: it records every change you make to your code as a snapshot (a commit), and because every developer has the full history on their own machine, nobody depends on one central server just to see what happened last month. The core loop — git init/clone, edit files, git add to stage changes, git commit to save a snapshot, git log/diff to inspect history — is the single most-used skill in professional software, full stop.

Real-world example: when an engineer's laptop is lost or their hard drive fails mid-sprint, they lose nothing — they clone the company's repository onto a new machine and the entire project history, down to every commit, is right there.

Branching & Branch Naming Conventions

A branch is an isolated line of development — you can build a feature, break things, and experiment freely without touching the working code everyone else depends on. Teams adopt naming conventions like feature/, bugfix/, hotfix/ prefixes so that anyone glancing at the repo instantly understands what a branch is for.

Real-world example: a checkout platform might have a branch named feature/paymongo-webhook-retry sitting alongside hotfix/cart-total-rounding-bug — the naming alone tells the whole team which one is routine and which needs eyes today.

Merging vs Rebasing

Both combine work from one branch into another, but they tell a different story afterward. A merge keeps the full, real history. A rebase rewrites your branch's commits so they look like they were built directly on top of the latest main, producing a clean, linear history but losing the record of exactly when the branch diverged.

Real-world example: many companies configure GitHub's 'squash and merge' button as the only allowed merge strategy, so every feature — no matter how many messy work-in-progress commits it had — lands on main as one clean commit.

Resolving Merge Conflicts

A conflict happens when two branches change the same lines of the same file and Git can't automatically decide which version is correct, so it stops and asks you to choose. This is completely normal on any team of more than one person.

Real-world example: two engineers both edit the same shipping-fee calculation in the same week; when the second person's PR is merged, Git flags the conflicting lines, and the two quickly hop on a call to agree on the final logic.

.gitignore & Keeping Secrets Out of Git

A .gitignore file tells Git which files to never track — build output, dependency folders, IDE settings, and critically, files holding passwords or API keys. Once a secret is committed, it's in the project's history forever unless the history itself is rewritten.

Real-world example: a startup once nearly leaked a live database password because a .env file was committed by accident; afterward, every new repo was required to start from a template with a strict .gitignore plus automated secret-scanning.

Undoing Changes: Stash, Reset & Revert

git stash temporarily shelves uncommitted changes. git reset rewrites history — powerful, but dangerous the moment anyone else has already pulled that history. git revert instead creates a brand-new commit that undoes an earlier one, leaving the original history intact and auditable.

Real-world example: when a bad deploy reaches production, the team runs git revert on the merge commit instead of force-pushing a reset, so the fix is visible in history and nobody else's work gets silently broken.

Commit Message Conventions (Conventional Commits)

A structured format like feat: add coupon validation or fix: correct tax rounding error makes history scannable at a glance, and tooling can read that structure to auto-generate changelogs or decide version bumps.

Real-world example: the Angular team's commit convention is well known precisely because their release notes and semantic version numbers are generated automatically from commit messages.

Pull Requests & Code Review

A pull request (PR) is a formal proposal to merge one branch into another: before code ever reaches main, at least one other engineer reviews it, asks questions, and can request changes. This is where bugs get caught before customers ever see them.

Real-world example: almost every tech company requires at least one approving review before a PR can merge — a new hire's first real lesson in the codebase's conventions often comes from review comments on their first PR.

GitHub Issues & Project Boards

Issues track individual bugs, features, or tasks with discussion attached directly to the code they concern; Project boards arrange issues into Kanban-style columns. Linking a PR to an issue lets merging it automatically close the issue.

Real-world example: a small startup runs its whole two-week sprint planning through a GitHub Project board — product, design, and engineering all look at the same board instead of a separate tracker that drifts out of sync.

Branching Strategy: Git Flow vs GitHub Flow / Trunk-Based

Git Flow uses long-lived develop, release, and hotfix branches and suits scheduled, versioned releases. GitHub Flow / trunk-based keeps a single main branch always deployable, with short-lived feature branches merged frequently — the norm at fast-moving SaaS companies.

Real-world example: a company shipping a public SaaS product might deploy from main dozens of times a day behind feature flags, while a bank's core-banking team still uses formal Git Flow release branches for compliance.

Branch Protection Rules & Required CI Checks

GitHub lets a repo owner lock the main branch so nobody — regardless of seniority — can push directly to it; every change must go through a PR, pass required checks, and get required approvals.

Real-world example: a company configures main so that even its own CTO cannot push directly to it — every change must pass CI and get an approving review first, preventing one bad Friday-afternoon push from breaking the site.

A quick forward-reference: the "required checks" mentioned above are usually written as **GitHub Actions** workflows — you'll write these yourself later in the roadmap once automated testing and CI/CD are covered.

Authentication: SSH Keys & Personal Access Tokens

SSH keys let you authenticate to GitHub without typing a password on every push. Personal Access Tokens (PATs) serve scripts and CI systems, and can be scoped to specific permissions and revoked individually.

Real-world example: a CI pipeline authenticates to pull a private repository using a scoped Personal Access Token rather than any engineer's personal login, so it can be revoked instantly without locking out a real person's account.

Suggested Project — Detailed Instructions

1 Repo From Day One

Steps: git init in your first mini-project folder → commit after every working increment → write a one-line README → push it to a new GitHub repo

2 Solo Feature Branch + PR

Steps: create a feature/ branch for one concept project → commit your work there → open a PR against your own main → read your own diff like a reviewer would, then merge

3 Conventional Commit Practice

Steps: make 5 small changes to any project → commit each with a feat:, fix:, refactor:, or chore: prefix → run git log --oneline to review the result

4 Merge Conflict Simulation

Steps: edit the same file/line on two branches → try merging one into the other → read the conflict markers → resolve manually and commit the resolution

5 Branch-Protected Practice Repo

Steps: create a new GitHub repo → turn on branch protection for main → require a PR and one passing check before merge → confirm a direct push to main is now blocked

6 GitHub CLI Drill [install the gh CLI → authenticate with gh auth login → open and merge a PR entirely from the terminal with gh pr create and gh pr merge → list open issues with gh issue list]

Steps: NEW

Study Resources

- Pro Git Book (free, official) — <https://git-scm.com/book/en/v2>
- Git official documentation — <https://git-scm.com/doc>
- GitHub Docs — <https://docs.github.com/en>
- Learn Git Branching (interactive, visual) — <https://learngitbranching.js.org/>
- GitHub CLI documentation — <https://cli.github.com/manual/>

4 · Git Commands Reference

The commands behind every topic above, grouped by what you're trying to do

■ **Careful:** `git reset --hard` permanently discards uncommitted changes — there is no undo. Always double-check `git status` before running it, and prefer `git revert` on any commit that's already been pushed or shared.

Command	What it does
<code>git init</code>	Turn the current folder into a new Git repository
<code>git clone</code>	Copy a remote repository (and its full history) to your machine
<code>git status</code>	Show what's staged, unstaged, and untracked right now
<code>git add</code> / <code>git add .</code>	Stage a specific file, or stage every changed file
<code>git commit -m "message"</code>	Save a snapshot of staged changes with a message
<code>git log</code> / <code>git log --oneline</code>	View commit history, full or condensed to one line each
<code>git diff</code>	Show unstaged changes line by line
<code>git branch</code>	List local branches; <code>git branch <name></code> creates a new one
<code>git checkout -b</code> / <code>git switch -c</code>	Create and switch to a new branch
<code>git merge</code>	Merge the named branch into your current branch
<code>git rebase</code>	Replay your commits on top of the named branch
<code>git stash</code> / <code>git stash pop</code>	Shelve uncommitted changes, then restore them later
<code>git reset --soft/--mixed/--hard</code>	Move the branch pointer back; <code>--hard</code> discards changes permanently — see warning above
<code>git revert</code>	Create a new commit that undoes a previous one, safely
<code>git push</code> / <code>git push -u origin</code>	Upload commits to the remote; <code>-u</code> sets the default upstream
<code>git pull</code>	Fetch and merge the remote branch into your current one
<code>git fetch</code>	Download remote changes without merging them yet
<code>git remote -v</code>	List the remote repositories this local repo is connected to
<code>git tag</code>	Mark a specific commit (e.g. a release) with a permanent label
<code>git cherry-pick</code>	Apply one specific commit from another branch onto yours
<code>git blame</code>	Show who last changed each line of a file, and in which commit
<code>git log --grep="text"</code>	Search commit messages for a keyword
<code>gh pr create</code> / <code>gh pr merge</code>	Create or merge a pull request from the terminal (GitHub CLI)

Pin this page. Every command here maps directly back to a topic in the Git & GitHub Essentials section above — if a command feels unfamiliar, that's the topic to re-read.

5 · UML Basics

Drawing your design before you code it · the shared language of software structure

NEW IN THIS REVISION: Project 1 revised: the original had students diagram inheritance and composition before OOP is taught in Phase 1. It now explicitly scopes the diagram to only association, since that's all Part 0 has covered so far.

UML (Unified Modeling Language) is a standardized set of diagram types for visualizing how a system is structured and how it behaves, before or alongside writing the code. You won't use every UML diagram type day to day, but class diagrams and sequence diagrams in particular come up constantly — in design docs, in whiteboard interviews, and in onboarding new teammates to a codebase.

Subtopics covered: class diagrams · use case diagrams · sequence diagrams · activity diagrams · ER diagrams

Class Diagrams

- Boxes represent classes: name, attributes, and methods in three stacked sections
- Relationship lines: association, aggregation (hollow diamond), composition (filled diamond), inheritance (hollow triangle arrow) — **aggregation, composition, and inheritance lines will make more sense once Phase 1 introduces OOP; for now focus on plain association**
- Multiplicity notation (1, 0..1, *, 1..*) describing how many of one class relate to another

Use Case Diagrams

- Actors (stick figures) representing users or external systems
- Ovals representing actions the system supports ("Place Order", "Cancel Subscription")
- Good for scoping a feature with a non-technical stakeholder before any code exists

Sequence Diagrams

- Vertical lifelines representing objects/services, time flowing top to bottom
- Horizontal arrows representing method calls or messages between them
- Extremely useful for mapping out an API request across Controller → Service → Repository → DB

Activity Diagrams

- Flowchart-style diagram for business logic: start node, decision diamonds, end node
- Good for modeling a multi-step process like checkout or order fulfillment

ER Diagrams (Entity-Relationship)

- Not strictly UML, but the same visual thinking applied to database schemas
- Entities as tables, attributes as columns, lines showing foreign key relationships and cardinality
- The natural bridge into Part A's Databases topic — draw this before writing a single CREATE TABLE

Suggested Project — Detailed Instructions

1 Class Diagram: ShopFlow Product & Cart (association only) [list Product and Cart as two plain classes with attributes only, no inheritance yet → connect them with a simple association line and multiplicity (one Cart relates to many CartItems) → revisit this same diagram after Phase 1 and add inheritance/composition once you've learned them → then implement the code and compare]

Steps: REVISED

2 Use Case Diagram: Checkout Flow

Steps: draw a Customer actor → add ovals for Browse, Add to Cart, Apply Coupon, Checkout → connect each to the actor → add a Payment Provider as a second actor for the payment use case

3 Sequence Diagram: Login Request

Steps: draw lifelines for Client, Controller, Service, Repository, DB → add the login request arrow from Client → add each internal call in order → add the JWT response arrow back to Client

4 Activity Diagram: Order Fulfillment

Steps: add a start node → add decision diamonds for payment success/failure → add activity boxes for each state transition → add an end node for both success and failure paths

5 ER Diagram: ShopFlow Schema

Steps: list entities: users, products, categories, orders, order_items → add key attributes to each → draw foreign key lines between related entities → mark one-to-many vs many-to-many relationships

Study Resources

- UML Diagrams reference (uml-diagrams.org) — <https://www.uml-diagrams.org/>
- OMG: Official UML Specification — <https://www.omg.org/spec/UML/>
- PlantUML documentation (text-based UML) — <https://plantuml.com/>

You already track your roadmap on Trello and sketch planning diagrams in Mermaid AI — Mermaid's class and sequence diagram syntax is a fast, text-based way to practice everything in this section without switching tools.

6 · Dev Tools & Apps Roundup

The apps that sit around your code, not in it — refreshed for 2026

You don't need every tool below on day one — this is a reference to come back to as each need shows up naturally through the roadmap.

Category	Tools
IDE / Editor	IntelliJ IDEA (Java/Spring Boot), VS Code (frontend, general-purpose)
Java Version Mgmt	SDKMAN (macOS/Linux) for installing and switching JDK versions
API Testing	Postman, Insomnia — manually test endpoints before wiring up the frontend
Database Client	DBeaver or pgAdmin (PostgreSQL), MongoDB Compass (MongoDB)
Diagramming	Mermaid (text-based, fast), draw.io / diagrams.net, Lucidchart
Project Tracking	Trello, GitHub Projects, Notion
Code Quality	SonarLint (in-IDE), ESLint + Prettier (JS/TS)
Design / Mockups	Figma — useful once Part B's frontend work starts needing a visual target
Terminal Enhancement	WSL2 (Windows), Oh My Zsh (Unix) — a real Unix shell plus nicer prompts and shortcuts
Git from the Terminal	GitHub CLI (gh) — create/merge PRs, manage issues, without leaving the terminal
API Documentation	Swagger/OpenAPI (auto-generated from your Spring Boot endpoints)
AI Coding Assistants	GitHub Copilot, Claude Code — useful once fundamentals are solid; leaning on them before syntax

NEW IN THIS REVISION: Added rows: **Java Version Mgmt** (SDKMAN), **Git from the Terminal** (GitHub CLI), and **AI Coding Assistants** — all standard parts of a 2026 developer's toolkit. WSL2 replaces Git Bash as the primary Windows recommendation.

Your current setup — IntelliJ IDEA, Trello for roadmap tracking, Mermaid AI for planning diagrams, and SonarLint for code quality — already covers the IDE, Project Tracking, Diagramming, and Code Quality rows above. The rest of this table fills in as Part A (Databases), Part B (Figma for frontend mockups), and Part C (API testing, documentation, and later Docker) come up.

Looking ahead: Docker and cloud deployment aren't part of Part 0's scope, but in short — Docker packages your app and everything it needs to run into one portable unit, which is why a Unix-style terminal (WSL2, not CMD) pays off now: Docker's own tooling assumes one.

Part 0 — Foundations — Progress Tracker (Revised)

Check off each topic once you've read its concept sections, built all hands-on projects, and folded the result into ShopFlow where indicated.

- 0 · Environment Setup (NEW)
 - 1 · Terminal & Command Line Basics
 - 2 · Java Anatomy & Syntax (expanded)
 - 3 · Git & GitHub Essentials
 - 4 · Git Commands Reference
 - 5 · UML Basics
 - 6 · Dev Tools & Apps Roundup
-

Summary of Changes in This Revision

- **Reordered:** Terminal moved before Java Anatomy (Java's own projects require it)
- **Added:** Environment Setup as a new topic 0
- **Added to Java Anatomy:** arrays, exception handling, Scanner/console input, reading stack traces & debugging
- **Added projects:** Array & Scanner Quiz App, Stack Trace Autopsy, GitHub CLI Drill, First Boot Checklist
- **Revised project:** Command-Line Calculator now uses real exception handling instead of a manual zero-check
- **Revised project:** UML class diagram scoped to association only, since inheritance/composition come in Phase 1
- **De-emphasized:** Windows CMD/PowerShell — now a brief fallback note, with WSL2 as the primary Windows recommendation
- **Added warning:** git reset --hard flagged directly in the commands reference table, not just in prose
- **Refreshed tools:** SDKMAN, GitHub CLI, WSL2, and AI coding assistants added to Dev Tools & Apps Roundup
- **Refreshed resources:** Baeldung, Java SE API docs (Javadoc), Learn Git Branching, GitHub CLI docs, WSL install guide added